VUPTH

Product Requirements Document

BACKEND — API Reference

Versão	Data	Testes	Deploy
v14	Mai 2026	120 pass / 0 fail	Render (homolog) · Railway (prod)

1. Stack e Infraestrutura

Componente	Detalhe
Linguagem	Java 21
Framework	Spring Boot 3.3
Banco de dados	PostgreSQL 16
Migrations	Liquibase (changesets versionados)
Autenticação	JWT RS256 (chaves assimétricas RSA)
Base package	com.ospaulos.vupth
CI/CD	GitHub Actions → dev→Render (homolog) → main→Railway (prod)
Containerização	Docker (alpine JRE 21)

1.1 URLs

Ambiente	URL Base
Homologação (Render)	https://vupth-backend-ia.onrender.com
Produção (Railway)	https://vupth-backend-ia-production.up.railway.app

⚠️ *Render hiberna instâncias gratuitas após inatividade. O primeiro request pode demorar ~30s para acordar o servidor.*

2. Autenticação — JWT RS256

2.1 Tokens

Token	Detalhe
Access token	JWT RS256 · expira em 1 hora · contém: sub, email, roles, exp
Refresh token	Opaque token · expira em 7 dias · rotativo (cada uso emite novo par)
Header	Authorization: Bearer <access_token>
Revogação	POST /auth/logout invalida o refresh_token. Access_token expira naturalmente.

2.2 Endpoints de Auth

Método	Endpoint	Auth	Retorno
`POST`	/auth/register	Público	201 — { id, email, created_at, user_roles }
`POST`	/auth/login	Público	200 — { user, tokens: { access_token, refresh_token, expires_in } }
`POST`	/auth/refresh	Público	200 — novo access_token + refresh_token
`POST`	/auth/logout	Bearer	204 — No Content

2.3 Sistema de Roles

Roles são acumulativas — um usuário pode ter CLIENT + OWNER + PROFESSIONAL simultaneamente.

Role	Como é concedida	Endpoint que concede
`CLIENT`	Automática no registro	POST /auth/register
`OWNER`	Ao criar establishment	POST /establishment/quick-create
`PROFESSIONAL`	Ao criar perfil profissional	POST /professional/quick-create ou /professional/register

⚠️ *CRITICAL: Após ganhar nova role, o cliente DEVE chamar POST /auth/refresh. O token antigo não reflete a nova role e endpoints retornarão 403.*

3. Endpoints Implementados

3.1 Profile — /profile

Todos requerem Bearer token. O userId é extraído do JWT — nunca do body.

Método	Endpoint	Role	HTTP	Observação
PATCH	/profile/general-infos	CLIENT	200	Todos os campos opcionais
PATCH	/profile/update-email/me	CLIENT	200	—
PATCH	/profile/update-password/me	CLIENT	200	—
DELETE	/profile/delete/me	CLIENT	204	Exclui conta permanentemente

PATCH /profile/general-infos — Campos

Campo	Tipo	Req	Validação
first_name	String	—	Qualquer string não-vazia
last_name	String	—	—
phone	String	—	—
birth_date	LocalDate	—	Formato: YYYY-MM-DD · mínimo 10 anos · máximo 120 anos
gender	Enum	—	FEMALE | MALE | NON_BINARY | OTHERS
profile_image_url	String	—	URL da foto

⚠️ *Body vazio {} → 400 NO_FIELDS_TO_UPDATE. Campo não reconhecido → 400 UNKNOWN_PROFILE_FIELD.*

3.2 Establishment — /establishment

Método	Endpoint	Role	HTTP	Obs
POST	/establishment/quick-create	CLIENT	201	Concede OWNER
POST	/establishment/create	OWNER	201	Criar adicional
GET	/establishment	Público	200	?name= ?serviceType=
GET	/establishment/id/{id}	Público	200	—

Método	Endpoint	Role	HTTP	Obs
GET	/establishment/id/{id}/professionals-with-queue	Público	200	Polling principal
PATCH	/establishment/{id}/open-status	OWNER	200	—
POST	/establishment/{id}/service-items	OWNER	201	—
POST	/establishment/{id}/professionals	OWNER	201	Adiciona prof existente

POST /establishment/quick-create — Body

```
{ "name": "Barbearia Top", "serviceType": "BARBER" }
```

POST /establishment/{id}/service-items — Body

```
{ "serviceItemType": "CORTE_MAQUINA", "customName": "Corte Degradê", "price": 35.00, "estimatedMinutes": 30 }
```

POST /establishment/{id}/professionals — Body (Owner adiciona prof existente)

```
{ "professionalId": "uuid-do-profissional" }
```

⚠️ *Owner só pode adicionar profissional ao próprio establishment. Establishment alheio → 403 ESTABLISHMENT_ACCESS_DENIED.*

⚠️ *Profissional já afiliado → 409 PROFESSIONAL_ALREADY_AFFILIATED (MVP: 1 establishment por vez).*

3.3 Professional — /professional

Método	Endpoint	Role	HTTP	Observação
POST	/professional/register	CLIENT	201	Cria sem establishment
POST	/professional/quick-create	CLIENT	201	Cria e vincula ao establishment
POST	/professional/affiliate	PROFESSIONAL	200	Vincula a um establishment
GET	/professional/{id}/service-items	Público	200	—

POST /professional/register — Body

```
{ "name": "João Silva", "specialty": "BARBER" }
```

POST /professional/quick-create — Body (solo shop)

```
{ "name": "João Silva", "establishmentId": "uuid", "specialty": "BARBER" }
```

POST /professional/affiliate — Body

```
{ "establishmentId": "uuid" }
```

3.4 Attendance — /attendances

Método	Endpoint	Role	HTTP	Observação
GET	/attendances/{id}	Autenticado	200	Qualquer role
POST	/attendances	CLIENT	201	clientId vem do JWT
PUT	/attendances/{id}/cancel	CLIENT	200	Só SCHEDULED
DELETE	/attendances/{id}/leave-queue	CLIENT	200	—
PATCH	/attendances/{id}/start	PROFESSIONAL	200	—
PATCH	/attendances/{id}/finish	PROFESSIONAL	200	—
PATCH	/attendances/{id}/pause	PROFESSIONAL	200	Só IN_PROGRESS
PATCH	/attendances/{id}/resume	PROFESSIONAL	200	Só PAUSED

POST /attendances — Body

```
{ "professionalID": "uuid", "establishmentId": "uuid", "serviceItemIds": ["uuid"], "scheduledTime": "2026-12-31T10:00:00+00:00" }
```

⚠️ *clientId no body é IGNORADO — backend usa o userId do JWT. scheduledTime deve ser ISO-8601 com offset, nunca timestamp numérico.*

⚠️ *Cliente só pode cancelar SCHEDULED. Tentar cancelar IN_PROGRESS, PAUSED ou FINISHED → 403.*

3.5 Queue — /professional-queue e /client-queue

Método	Endpoint	Role	HTTP	Obs
POST	/professional-queue/add-quest-attendance	Público	201	Guest sem conta
DELETE	/professional-queue/{id}/remove	PROFESSIONAL	200	Remove da fila
GET	/client-queue/{profId}/queue	Público	200	?establishmentId=
GET	/client-queue/{attendanceId}/wait-time	Público	200	?clientId=

3.6 Address — /establishments e /users/me

Método	Endpoint	Role	HTTP	Obs
POST	/establishments/{id}/addresses	OWNER	201	zipCode: 8 dígitos puros
POST	/users/me/addresses	CLIENT	201	—
GET	/users/me/addresses	CLIENT	200	—
GET	/users/me/addresses/main	CLIENT	200	—
PATCH	/users/me/addresses/{id}	CLIENT	200	—
PATCH	/users/me/addresses/{id}/main	CLIENT	200	Define endereço principal
DELETE	/users/me/addresses/{id}	CLIENT	200	—

⚠️ *zipCode armazenado como 8 dígitos puros sem hífen: 01310100. Regex: ^\d{8}$. @Column(length=8).*

---

4. Regras de Negócio

4.1 Profissional

- MVP: profissional só pode estar vinculado a 1 establishment por vez
- quick-create: cria e já vincula — para solo shop (owner + prof da própria loja)
- register: cria SEM establishment — para profissional que aguarda convite do owner
- affiliate: vincula profissional já existente — requer establishment == null
- Já afiliado → 409 PROFESSIONAL_ALREADY_AFFILIATED
- Profissional inativo (is_available: false) não aparece na listagem pública

4.2 Owner

- Owner valida posse do establishment em toda operação OWNER
- Owner diferente do dono do establishment → 403 ESTABLISHMENT_ACCESS_DENIED

4.3 Attendance

- clientId vem SEMPRE do JWT — campo clientId no body de POST /attendances é ignorado
- Cliente não pode cancelar IN_PROGRESS, PAUSED ou FINISHED → 403
- Cliente não pode cancelar CANCELLED → 409

- Só SCHEDULED pode ser cancelado pelo cliente
- Profissional só pausa IN_PROGRESS → status vira PAUSED
- Profissional só retoma PAUSED → status vira IN_PROGRESS
- CANCELLED usa attendance.cancel() (método de domínio) — nunca setStatus() diretamente
- AttendanceNotFoundException para UUID inexistente → 404 (nunca 500)

4.4 Queue

- getQueueForProfessional usa query direta ao banco (não lista LAZY em memória)
- Isso garante que guests recém-adicionados aparecem imediatamente na fila
- enqueueGuestAttendance não manipula lista em memória — delegate para getQueueOverview

4.5 Validações Críticas

- zipCode: regex ^\d{8}$ — VARCHAR(8) — sem hífen ou ponto
- scheduledTime: ISO-8601 com offset obrigatório — timestamp numérico → 400
- AttendanceStatus.CANCELLED: duplo L — nunca CANCELED
- Enums sem prefixo ROLE_ no banco: CLIENT, OWNER, PROFESSIONAL
- birth_date: mínimo 10 anos, máximo 120 anos — datas futuras → 400 INVALID_BIRTH_DATE

---

5. Padrão de Erros

Todos os erros da API seguem a estrutura:

```
{ "status": 422, "message": "Field validation error", "code": "VALIDATION_ERROR",
  "fieldErrors": { "email": "must be a valid email" }, "timestamp": "2026-05-
02T10:00:00Z" }
```

5.1 Mapeamento de Error Codes

HTTP	code	Quando ocorre
400	`INVALID_REQUEST_BODY`	UUID mal formado, tipo errado no body, scheduledTime numérico
400	`NO_FIELDS_TO_UPDATE`	PATCH /profile/general-infos com body vazio {}
400	`UNKNOWN_PROFILE_FIELD`	Campo desconhecido em PATCH /profile/general-infos
400	`INVALID_BIRTH_DATE`	Data futura ou fora do range [10–120 anos]
400	`INVALID_PATH_VARIABLE`	UUID inválido no path (@PathVariable)
401	—	Token ausente, inválido ou expirado

HTTP	code	Quando ocorre
403	`ESTABLISHMENT_ACCESS_DENIED`	Owner tentando acessar establishment de outro
403	—	Role insuficiente para o endpoint
404	`ATTENDANCE_NOT_FOUND`	Attendance UUID inexistente
404	`PROFESSIONAL_NOT_FOUND`	Profissional UUID inexistente
404	`ESTABLISHMENT_NOT_FOUND`	Establishment UUID inexistente
409	`PROFESSIONAL_ALREADY_AFFILIATED`	Profissional já tem establishment (MVP)
422	`VALIDATION_ERROR`	Falha em @Valid — ver fieldErrors no body

6. Enums da API

6.1 ServiceType

Valor	Descrição
`BARBER`	Barbearia
`SALON_BEAUTY`	Salão de beleza
`EYEBROW`	Design de sobrancelha
`MANICURE`	Manicure / pedicure

6.2 ServiceItemType

Valor	Estabelecimentos compatíveis
`CORTE_MAQUINA`	BARBER
`RECORTE_PONTAS`	SALON_BEAUTY
`PINTURA`	BARBER, SALON_BEAUTY
`MANICURE`	MANICURE
`PEDICURE`	MANICURE

6.3 AttendanceStatus

Valor	Quando ocorre
`SCHEDULED`	Na fila, aguardando ser chamado

Valor	Quando ocorre
`IN_PROGRESS`	Sendo atendido agora
`PAUSED`	Atendimento pausado temporariamente
`FINISHED`	Atendimento concluído
`CANCELLED`	Cancelado — duplo L — nunca CANCELED

6.4 OpenStatus

Valor	Descrição
`OPEN`	Aberto e recebendo clientes
`CLOSED`	Fechado
`TEMPORARILY_CLOSED`	Fechado temporariamente

7. Pendências — Roadmap

7.1 Próximos Endpoints (pós-MVP)

Método	Endpoint	Descrição
`DELETE`	/establishment/{id}	Owner deleta establishment
`GET`	/owner/establishments	Owner lista próprios establishments
`DELETE`	/establishment/{id}/professionals/{profId}	Owner remove profissional do establishment
`POST`	/professional-queue/{id}/intervals/unexpected	Pausa inesperada
`POST`	/professional-queue/{id}/intervals/recurring	Pausa recorrente (almoço, etc.)
`GET`	/professional/{id}/intervals	Lista intervalos do dia

7.2 Checklist de Erros Já Cometidos (não repetir)

- NUNCA zipar o projeto inteiro — só arquivos modificados
- str_replace substituindo só imports → duplicate class. Sempre substituir de package até o último }
- Enum values sem prefixo ROLE_ no seed: CLIENT, OWNER (não ROLE_CLIENT)
- findById → findByIdEager nos mocks de teste — rodar grep após adicionar findByIdEager
- CANCELLED duplo L — nunca CANCELED

- findByIdEager após save() nos métodos create*
- JOIN FETCH em queries usadas em endpoints públicos (Lazy fora de sessão)
- Rota no SecurityConfig = método no controller — verificar os dois juntos
- Dois métodos com mesmo @Order no JUnit — comportamento indefinido
- Refresh script sem guard: if (newToken && newToken.length > 20) antes de set
- zipCode nos testes: String.format('%08d', ...) — 8 dígitos puros
- getQueueForProfessional: NÃO usar professional.getScheduledAttendances() — usar query direta ao banco
- @JsonIgnoreProperties(ignoreUnknown=false) em records Java com @JsonProperty quebra desserialização — usar @JsonAnySetter